



UNIVERSIDAD PRIVADA DEL VALLE

FACULTAD: FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA: INGENIERÍA DE SISTEMAS INFORMÁTICOS

MATERIA: TECNOLOGÍAS EMERGENTES I

LG15 - TOLERANCIA A FALLOS Y REPLICACIÓN DE DATOS EN HDFS

MATERIA: Tecnologías Emergentes I

DOCENTE: Ing. Efrain Luna Mamani

GRUPO: A

ESTUDIANTES: Anthony Alberth Arevalo Beltran

Raúl Jiménez Camacho

Hiber Leandro Veizaga Crezpo

FECHA PRESENTACIÓN INFORME: 11/10/2026

1. Objetivo

Implementar un entorno distribuido Hadoop HDFS compuesto por 1 NameNode y 3 DataNodes, configurando un factor de replicación mínimo de 2, para analizar experimentalmente el comportamiento de HDFS ante la caída de un DataNode mediante la carga y recuperación de datasets de 1 GB y 10 GB.

El estudiante deberá demostrar, mediante evidencias verificables, la distribución de los bloques, las réplicas existentes y el comportamiento del sistema antes, durante y después de la caída de un DataNode.

2. Requerimientos mínimos

Cada grupo deberá implementar un entorno con:

Componente	Cantidad mínima
NameNode	1
DataNode	3
Factor de replicación	≥ 2
Dataset 1	1 GB
Dataset 2	10 GB
Pruebas de fallo	2

3. Identificación del grupo

Para garantizar que la práctica fue realizada por el grupo y no simplemente reproducida de otra implementación, **todos los directorios relacionados con la práctica deberán utilizar el número de grupo.**

Por ejemplo, si el grupo es el **Grupo 05**:

/grupo05/

Y dentro:

/grupo05/dataset_1GB/

/grupo05/dataset_10GB/

/grupo05/resultados/

/grupo05/evidencias/

También deberán utilizarse nombres identificables para los archivos:

dataset_grupo05_1GB

dataset_grupo05_10GB

El número de grupo deberá aparecer además en las evidencias solicitadas.

4. Prueba de fallo con cada dataset (con la de 1GB y con la e 10GB)

Esta será una de las partes **más importantes de la práctica.**

Antes de detener un DataNode, deberán registrar el estado inicial.

Estado inicial

Por ejemplo:

DN1 → Activo

DN2 → Activo

DN3 → Activo

Y deberán identificar dónde se encuentran las réplicas del dataset.

Posteriormente, deberán **detener solamente un DataNode**.

Por ejemplo:

DN2 → FALLA

Deberán demostrar

¿Qué ocurre con el dataset de 1 GB?

Deberán verificar:

- Estado del DataNode.
- Estado del archivo.
- Estado de los bloques.
- Número de réplicas disponibles.
- Si el archivo continúa siendo accesible.
- Si se puede leer el archivo.
- Si HDFS identifica bloques subreplicados.
- Si HDFS inicia o no un proceso de replicación.

Por ejemplo:

```
hdfs dfsadmin -report
```

y:

```
hdfs fsck /grupo05/dataset_1GB -files -blocks -locations
```

-

5. Análisis

Deberán responder:

Antes de la falla

- ¿Cuántos bloques tenía el archivo?

El archivo de 1 GiB tenía 8 bloques y el archivo de 10 GiB tenía 80 bloques. Esto se debe a que HDFS estaba utilizando bloques de 128 MiB.

- **¿Cuántas réplicas tenía cada bloque?**

Cada bloque tenía 2 réplicas, porque el clúster fue configurado con `dfs.replication=2`. Antes de la falla, `fsck` mostraba una replicación promedio de 2.0.

- **¿En qué DataNodes estaban?**

Las réplicas estaban distribuidas entre `DataNode1`, `DataNode2` y `DataNode3`. No existían dos copias completas del archivo almacenadas en dos nodos fijos; HDFS distribuía las réplicas de cada bloque entre distintos `DataNodes`. En la prueba de 1 GiB, 4 de los 8 bloques tenían una réplica en DN2. En la prueba de 10 GiB, 60 de los 80 bloques tenían una réplica en DN2.

Durante la falla

- **¿Qué DataNode fue detenido?**

R.- Se detuvo intencionalmente `DataNode2`.

- **¿Qué bloques estaban almacenados en él?**

R.- Antes de detenerlo, DN2 participaba en el almacenamiento de réplicas de 4 bloques en el dataset de 1 GiB y de 60 bloques en el dataset de 10 GiB.

- **¿Existían otras réplicas?**

R.- Sí. Debido al factor de replicación 2, los bloques que tenían una réplica en DN2 también tenían otra réplica almacenada en otro `DataNode`. Por ello, la caída de DN2 no produjo bloques perdidos.

- **¿El archivo continuó disponible?**

R.- Sí. El archivo permaneció disponible porque HDFS pudo utilizar las réplicas almacenadas en los `DataNodes` supervivientes. En la prueba de 1 GiB se comprobó además la lectura completa del archivo durante la falla y el comando terminó con código de salida 0. En las comprobaciones `fsck` se observaron 0 bloques perdidos.

- **¿Qué cambió en HDFS?**

R.- El número de `DataNodes` disponibles pasó de 3 a 2. Después de detectar que DN2 había dejado de enviar heartbeats, el `NameNode` identificó las réplicas que ya no estaban disponibles e inició la recuperación automática.

Después de la falla

- **¿Qué estado reporta HDFS?**

R.- HDFS reportó los archivos como `HEALTHY`. Después de la recuperación no se observaron bloques perdidos.

- **¿Existen bloques subreplicados?**

R.- Durante la recuperación del dataset de 10 GiB se capturaron 26 bloques subreplicados (32.5 %). En el dataset de 1 GiB, cuando se tomó la evidencia, HDFS ya había terminado la recuperación y reportaba 0 bloques subreplicados y replicación promedio 2.0.

- **¿HDFS intenta recuperar automáticamente las réplicas?**

R.- Sí. El NameNode detecta los bloques que quedan por debajo del factor de replicación configurado y programa la creación de nuevas réplicas utilizando las copias que permanecen disponibles. La prueba de 10 GiB evidencia este proceso: DN2 tenía réplicas de 60 bloques antes de la falla y, cuando se capturó **fsck**, solamente quedaban 26 bloques subreplicados, lo que indica que la recuperación ya estaba en progreso.

- **¿En qué DataNode se generaron las nuevas réplicas?**

R.- Al estar DN2 fuera de servicio, las nuevas réplicas tuvieron que generarse utilizando los DataNodes supervivientes, DN1 y DN3. Para determinar exactamente qué bloques fueron replicados hacia DN1 y cuáles hacia DN3, habría que comparar las ubicaciones (locations) de cada bloque en los archivos **fsck** de antes y durante/después de la falla. Con la evidencia que tenemos no conviene afirmar un único DataNode específico sin hacer esa comparación.

Las siguientes preguntas son parte para la exposición del trabajo:

Pregunta 1 ¿Por qué HDFS puede continuar proporcionando el archivo aunque un DataNode haya fallado?

R.- Porque HDFS almacena réplicas de los bloques en diferentes DataNodes. Con un factor de replicación de 2, cada bloque dispone normalmente de dos réplicas. Si uno de los DataNodes falla, HDFS puede obtener el bloque desde otra réplica disponible.

Pregunta 2 ¿Qué habría ocurrido si el factor de replicación hubiera sido 1?

R.- Cada bloque tendría una única copia. Si el DataNode que contiene ese bloque falla, dicho bloque dejaría de estar disponible. Si el archivo necesita ese bloque, el archivo completo no podría leerse correctamente.

Pregunta 3 ¿Qué habría ocurrido si hubieran fallado **dos DataNodes simultáneamente**?

R.- Con un factor de replicación de 2, existe la posibilidad de que ambas réplicas de determinados bloques estuvieran precisamente en los dos DataNodes que fallaron. Esos bloques quedarían indisponibles y el archivo no podría leerse completamente.

Pregunta 4 ¿Por qué un factor de replicación de 2 no significa necesariamente que existan dos copias completas del archivo en dos DataNodes?

R.- Porque HDFS realiza la replicación por bloques y no por archivos completos. Las réplicas de los distintos bloques pueden distribuirse entre diferentes DataNodes.

Pregunta 5 ¿Cuál es la relación entre tamaño de bloque y número de bloques?

R.- El número de bloques depende aproximadamente de la siguiente fórmula:

Número de bloques = $\text{ceil}(\text{Tamaño del archivo} / \text{Tamaño del bloque})$

Con un tamaño de bloque de 128 MiB:

1 GiB → 8 bloques

10 GiB → 80 bloques

Pregunta 6 ¿Por qué el dataset de 10 GB permite observar un comportamiento más representativo de un sistema distribuido que el dataset de 1 GB?

R.- Porque genera una cantidad mayor de bloques. Esto produce más oportunidades de distribución y replicación entre los DataNodes y permite observar de mejor manera el comportamiento de almacenamiento distribuido de HDFS.

Pregunta 7 ¿Qué sucede con los bloques que tenían una réplica en el DataNode que falló?

R.- La réplica almacenada en el DataNode caído deja de estar disponible. El NameNode detecta la reducción de réplicas y puede ordenar la creación de nuevas réplicas a partir de las copias que permanecen disponibles en los DataNodes supervivientes.

Pregunta 8 ¿Quién administra la ubicación y replicación de los bloques: el cliente, el DataNode o el NameNode?

R.- El NameNode mantiene los metadatos relacionados con los archivos y bloques y coordina la replicación. Los DataNodes almacenan físicamente los bloques y ejecutan las operaciones indicadas por el NameNode.

Pregunta 9 ¿Qué diferencia existe entre **disponibilidad del archivo** y **cumplimiento del factor de replicación** después de una falla?

R.- La disponibilidad indica si todavía existen suficientes bloques para leer completamente el archivo. El cumplimiento del factor de replicación indica si cada bloque mantiene el número configurado de réplicas. Por lo tanto, un archivo puede continuar disponible aunque temporalmente algunos bloques tengan menos réplicas de las configuradas.

6. Comparación

El grupo deberá elaborar una comparación entre ambos experimentos.

Característica	Dataset 1 GB	Dataset 10 GB
Tamaño	1 GB	10 GB
Número de bloques	8 bloques	80 bloques
Factor de replicación	2	2
DataNode afectado	DN2 (datanode2)	DN2 (datanode2)
Bloques afectados	4 bloques tenían una réplica en DN2 antes de la falla	60 bloques tenían una réplica en DN2 antes de la falla
Bloques disponibles después de falla	8 bloques (todos), 0 bloques perdidos	80 bloques (todos), 0 bloques perdidos
¿Archivo accesible?	Sí. Se comprobó la lectura durante la falla	Sí. HDFS mantuvo 0 bloques perdidos
¿Existieron bloques subreplicados?	Al capturar fsck: 0 (0.0%); HDFS ya había recuperado las réplicas	Sí: 26 bloques (32.5%) durante la recuperación
Tiempo de recuperación	Muy rápido	Más lento

Nuevas réplicas	HDFS reconstruyó automáticamente las réplicas utilizando los nodos supervivientes DN1 y DN3	HDFS estaba reconstruyendo automáticamente las réplicas utilizando DN1 y DN3
------------------------	---	--

Repositorio propio de GitHub.

El repositorio deberá contener como mínimo:

- **README.md** con la descripción del proyecto, instalación, ejecución y resultados.
- Archivos de configuración utilizados (**docker-compose.yml**, Dockerfile, etc.).
- Scripts utilizados.
- Diagrama de arquitectura.
- Evidencias de la implementación y pruebas.

Requisitos

- El repositorio deberá ser **público**.
- El nombre deberá identificar al grupo, por ejemplo:
bigdata-numero_de_grupo
- Deberá incluir al menos **3 commits** que evidencien el avance del trabajo.

Entrega

El grupo deberá entregar únicamente la **URL de su repositorio GitHub**.